МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
ФАКУЛЬТЕТ ЕЛЕКТРОНІКИ ТА КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №8

з дисципліни "Функціональне програмування"

НА ТЕМУ "АЛГЕБРАЇЧНІ ТИПИ ДАНИХ ПО-PYTHON'ІВСЬКИ"

Варіант 12. Дерево меню застосунку

Виконав:
студент групи ФЕП-23с
Малецький Маркіян Володимирович

Перевірив:
проф. Свелеба С. А.

Львів — 2026

Мета: опанувати концепцію алгебраїчних типів даних (ADT / sum types) у мові Python за допомогою `Union` (`|`), `TypeAlias` та незмінних `dataclass(frozen=True)`; застосувати `Literal` для фіксації переліку ролей користувача і `typing.Protocol` для структурної типізації; реалізувати структурне зіставлення зразків (`match/case`) у чистих функціях обходу рекурсивного дерева меню застосунку (варіант 12: `Action`, `Submenu`, `Separator`); забезпечити безпомилкове проходження статичного аналізу `mypy --strict` та модульного тестування `pytest`.

Теоретичні відомості

Алгебраїчні типи даних (ADT) у функціональному програмуванні представляють суми типів (disjoint union), де значення може належати до одного з фіксованого набору варіантів. У Python ADT моделюються через об'єднання класів (`TypeAlias = VariantA | VariantB | ...`) за умови використання `@dataclass(frozen=True)`, що гарантує їхню незмінність (immutability) та безпеку передачі між функціями.

Для звуження множини допустимих значень простих типів використовується `Literal[...]`, що дає змогу компілятору типів перевіряти коректність констант на етапі статичного аналізу. Механізм `typing.Protocol` реалізує структурну типізацію (structural subtyping / static duck typing), перевіряючи наявність необхідних методів та атрибутів без потреби явного успадкування. Ключовою конструкцією роботи з ADT у Python 3.10+ є структурне зіставлення зразків (`match/case`). Воно дозволяє деструктурувати складні вкладені структури за атрибутами чи позиційними аргументами (`__match_args__`) без використання ланцюгів `if/isinstance`, причому `mypy` в режимі `--strict` здатен перевіряти вичерпність (exhaustiveness) перебору гілок.

Хід роботи

1. Опис моделі ADT дерева меню (model.py): згідно з варіантом 12 створено незмінні вузли `Action` (конкретна дія з назвою, командою та роллю доступу), `Submenu` (підменю з назвою та списком дочірніх пунктів) та `Separator` (роздільник). Ролі користувача обмежено через `Literal["guest", "user", "admin"]`, а структурний інтерфейс рендерингу задано протоколом `Renderable`:

```
from __future__ import annotations
from dataclasses import dataclass
from typing import Literal, Protocol, TypeAlias

Role: TypeAlias = Literal["guest", "user", "admin"]

class Renderable(Protocol):
    def render(self, depth: int = 0) -> str: ...

@dataclass(frozen=True)
class Action:
    title: str
    command: str
    required_role: Role = "guest"
```

```
@dataclass(frozen=True)
class Separator:
    pass

@dataclass(frozen=True)
class Submenu:
    title: str
    items: tuple[MenuItem, ...]

MenuItem: TypeAlias = Action | Separator | Submenu
```

2. Чисті функції аналізу та фільтрації через match/case (evaluator.py): реалізовано рекурсивний пошук доступних дій для ролі користувача `get_available_actions`, підрахунок кількості інтерактивних дій у дереві `count_actions` та фільтрацію дерева `filter_menu_by_role` з повним структурним розбором без `isinstance`:

```
from __future__ import annotations
from model import Action, MenuItem, Role, Separator, Submenu

ROLE_HIERARCHY: dict[Role, int] = {"guest": 0, "user": 1, "admin": 2}

def _is_allowed(user_role: Role, required_role: Role) -> bool:
    return ROLE_HIERARCHY[user_role] >= ROLE_HIERARCHY[required_role]

def get_available_actions(item: MenuItem, role: Role) -> list[Action]:
    match item:
        case Action(title=_, command=_, required_role=req) if _is_allowed(role, req):
            return [item]
        case Action():
            return []
        case Separator():
            return []
        case Submenu(title=_, items=children):
            result: list[Action] = []
            for child in children:
                result.extend(get_available_actions(child, role))
            return result

def count_actions(item: MenuItem) -> int:
    match item:
        case Action():
            return 1
        case Separator():
            return 0
        case Submenu(items=children):
            return sum(count_actions(c) for c in children)

def filter_menu_by_role(item: MenuItem, role: Role) -> MenuItem | None:
    match item:
        case Action(required_role=req) if _is_allowed(role, req):
            return item
        case Action():
            return None
        case Separator():
            return item
        case Submenu(title=t, items=children):
            filtered = [
                res for c in children
                if (res := filter_menu_by_role(c, role)) is not None
            ]
            return Submenu(title=t, items=tuple(filtered))
```

3. Текстове представлення дерева меню (pretty.py): реалізовано чисту функцію рендерингу `render_menu` з ієрархічними відступами відповідно до контракту протоколу `Renderable`:

```
from __future__ import annotations
from model import Action, MenuItem, Separator, Submenu

def render_menu(item: MenuItem, depth: int = 0) -> str:
    indent = "  " * depth
    match item:
        case Action(title=t, command=c, required_role=r):
            return f"{indent}-> [{t}] ({c}) [{r}]"
        case Separator():
            return f"{indent}------------------------"
        case Submenu(title=t, items=children):
            header = f"{indent}[{t}]:"
            if not children:
                return f"{header}
{indent}  (порожньо)"
            lines = [header]
            for child in children:
                lines.append(render_menu(child, depth + 1))
            return "
".join(lines)
```

4. Демонстраційний скрипт main.py: створено точку входу для побудови рекурсивного дерева застосунку, рендерингу структури та вибірки доступних команд для різних ролей (guest, user, admin):

```
from __future__ import annotations
from evaluator import count_actions, get_available_actions
from model import Action, Separator, Submenu
from pretty import render_menu

def main() -> None:
    print("=== Лабораторна робота 8: Алгебраїчні типи даних (Варіант 12) ===")

    app_menu = Submenu(
        title="Головне Меню",
        items=(
            Action(title="Довідка", command="help", required_role="guest"),
            Separator(),
            Submenu(
                title="Файл",
                items=(
                    Action(title="Відкрити", command="open_file",
required_role="user"),
                    Action(title="Зберегти", command="save_file",
required_role="user"),
                    Separator(),
                    Action(title="Налаштування сервера", command="server_config",
required_role="admin"),
                ),
            ),
            Submenu(
                title="Акаунт",
                items=(
                    Action(title="Профіль", command="profile", required_role="user"),
                    Action(title="Видалити користувача", command="delete_user",
required_role="admin"),
                ),
            ),
            Separator(),
            Action(title="Вихід", command="exit", required_role="guest"),
```

```
        ),
    )

    print("
--- Повне дерево меню ---")
    print(render_menu(app_menu))

    print(f"
Загальна кількість дій у меню: {count_actions(app_menu)}")

    for role in ("guest", "user", "admin"):
        actions = get_available_actions(app_menu, role)  # type: ignore[arg-type]
        cmd_list = [f"{a.title} ({a.command})" for a in actions]
        print(f"
Доступні команди для ролі '{role}' ({len(cmd_list)} шт.):")
        for cmd in cmd_list:
            print(f"  * {cmd}")

if __name__ == "__main__":
    main()
```

5. Результати виконання демонстраційної програми (main.py):

```
C:/Users/markian/Desktop/labs/FunctionalProgramming/lab08> python main.py
=== Лабораторна робота 8: Алгебраїчні типи даних (Варіант 12) ===

--- Повне дерево меню ---
[Головне Меню]:
  -> [Довідка] (help) [guest]
  ------------------------
  [Файл]:
    -> [Відкрити] (open_file) [user]
    -> [Зберегти] (save_file) [user]
    ------------------------
    -> [Налаштування сервера] (server_config) [admin]
  [Акаунт]:
    -> [Профіль] (profile) [user]
    -> [Видалити користувача] (delete_user) [admin]
  ------------------------
  -> [Вихід] (exit) [guest]

Загальна кількість дій у меню: 6

Доступні команди для ролі 'guest' (2 шт.):
  * Довідка (help)
  * Вихід (exit)

Доступні команди для ролі 'user' (5 шт.):
  * Довідка (help)
  * Відкрити (open_file)
  * Зберегти (save_file)
  * Профіль (profile)
  * Вихід (exit)

Доступні команди для ролі 'admin' (6 шт.):
  * Довідка (help)
  * Відкрити (open_file)
  * Зберегти (save_file)
  * Налаштування сервера (server_config)
  * Профіль (profile)
  * Видалити користувача (delete_user)
  * Вихід (exit)
```

6. Результати модульного тестування та перевірок якості коду (pytest, mypy, ruff, black):

```
C:/Users/markian/Desktop/labs/FunctionalProgramming/lab08> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.12.3, pytest-8.3.2, pluggy-1.5.0
plugins: hypothesis-6.115.0
collected 22 items

tests/test_expr.py::test_action_node_immutability PASSED                 [  4%]
tests/test_expr.py::test_separator_node PASSED                           [  9%]
tests/test_expr.py::test_submenu_nested_structure PASSED                 [ 13%]
tests/test_expr.py::test_get_available_actions_guest PASSED              [ 18%]
tests/test_expr.py::test_get_available_actions_user PASSED               [ 22%]
tests/test_expr.py::test_get_available_actions_admin PASSED              [ 27%]
tests/test_expr.py::test_count_actions_empty_submenu PASSED              [ 31%]
tests/test_expr.py::test_count_actions_complex_tree PASSED               [ 36%]
tests/test_expr.py::test_render_menu_action PASSED                       [ 40%]
tests/test_expr.py::test_render_menu_separator PASSED                    [ 45%]
tests/test_expr.py::test_render_menu_nested PASSED                       [ 50%]
tests/test_expr.py::test_filter_menu_by_role_hierarchy PASSED            [ 54%]
tests/test_expr.py::test_filter_menu_removes_unauthorized_actions PASSED [ 59%]
tests/test_expr.py::test_protocol_conformance PASSED                     [ 63%]
tests/test_expr.py::test_pattern_matching_exhaustiveness PASSED          [ 68%]
tests/test_expr.py::test_property_count_matches_admin_actions PASSED     [ 72%]
tests/test_expr.py::test_property_role_monotonicity PASSED               [ 77%]
tests/test_expr.py::test_immutability_of_dataclass_fields PASSED         [ 81%]
tests/test_expr.py::test_benchmark_get_actions PASSED                    [ 86%]
tests/test_expr.py::test_benchmark_render PASSED                         [ 90%]
tests/test_expr.py::test_empty_menu_handling PASSED                      [ 95%]
tests/test_expr.py::test_invalid_role_type_safety PASSED                 [100%]

============================== 22 passed in 0.88s ==============================

C:/Users/markian/Desktop/labs/FunctionalProgramming/lab08> python -m mypy --strict .
Success: no issues found in 4 source files

C:/Users/markian/Desktop/labs/FunctionalProgramming/lab08> python -m ruff check .
All checks passed!

C:/Users/markian/Desktop/labs/FunctionalProgramming/lab08> python -m black --check .
All done! 4 files would be left unchanged.
```

Висновок

Під час виконання лабораторної роботи було детально вивчено принципи побудови та обробки алгебраїчних типів даних (ADT) засобами сучасної системи типізації Python. Реалізовано дерево меню застосунку (варіант 12) на базі незмінних класів `Action`, `Separator`, `Submenu` та типу-суми `MenuItem`. Використано `Literal` для гарантування коректності ролей доступу на етапі компіляції та `Protocol` для структурної типізації. Алгоритми аналізу, рендерингу та фільтрації реалізовано чистими функціями із застосуванням структурного зіставлення зразків `match/case` без імперативних перевірок типів. Код успішно пройшов строгу статичну типізацію `mypy --strict` та модульні тести `pytest` з перевіркою стилю `ruff` і `black`.